# Complete technical blueprint for a WhatsApp + voice AI SaaS platform

**Building a multi-tenant platform that deploys AI chat and voice agents for Mexican SMBs requires weaving together seven distinct infrastructure layers — tenant isolation, intelligent onboarding, marketplace orchestration, omnichannel conversation management, anti-hallucination guardrails, WhatsApp and telephony APIs, and intelligent model routing.** This blueprint synthesizes production-tested architecture patterns, database schemas, API integration flows, and cost models specific to the Mexican market, where **77 million WhatsApp users** (World Population Review) and **93% penetration** (Aurora Inbox) among internet users (Aurora Inbox) make it the dominant business communication channel.

The platform targets dental clinics, restaurants, salons, psychologists, and real estate agencies — industries where missed messages directly translate to lost revenue. Aurora Inbox (Monterrey-based) has validated this market with plans starting at **$1,800 MXN/month**, (Aurora Inbox) (aurorainbox) and Stammer.ai has proven the agency→sub-account white-label model at scale. (Stammer) This blueprint goes further: a complete AI agent factory where answering a questionnaire produces a working chat and voice agent in under 10 minutes.

---

## 1. Multi-tenant architecture built on Supabase with Row Level Security

The foundation is a **shared PostgreSQL database with `tenant_id` columns on every table, enforced by Supabase Row Level Security**. This approach supports 0–1,000+ tenants with minimal operational complexity, while Supabase provides auth, real-time subscriptions, pgvector for RAG, edge functions for webhook processing, and storage for knowledge base files — all in one platform.

**Three isolation strategies exist**, ranked by complexity: shared DB with tenant_id filtering (recommended for MVP through 1,000 tenants), schema-per-tenant (for enterprise compliance), and database-per-tenant (regulated industries only). (Bix-tech) The shared model works because RLS policies are enforced at the database engine level, (Supabase) (antstack) meaning even a bug in application code cannot leak data across tenants. (Redis) (WorkOS)

The core schema requires these primary tables: `tenants` (with plan limits, WhatsApp credentials, RFC for Mexican tax invoicing, and feature flags as JSONB), `agents` (per-tenant AI agent configurations including model selection, temperature, language, and timezone), `agent_prompts` (versioned system prompts with `is_active` flags for rollback), `knowledge_documents` and `document_sections` (RAG storage with pgvector embeddings), `conversations` and `messages` (channel-agnostic with sender_type discriminator for

customer/bot/human), `contacts` (phone number as primary identity key across WhatsApp and voice), and `usage_logs` for per-tenant billing attribution.

RLS implementation follows a critical pattern: a helper function `get_tenant_id()` extracts the tenant UUID from the JWT custom claim, and every table gets a policy like `USING (tenant_id = get_tenant_id())`. **Always index columns referenced in RLS policies**, test from the client SDK (not the SQL Editor, which bypasses RLS), and use the `service_role` key only server-side. For vector search isolation, a custom `match_documents_for_tenant()` RPC function adds a `WHERE tenant_id = p_tenant_id` clause before the cosine similarity calculation, ensuring embeddings from one business never surface for another.

Prompt management uses a **three-layer architecture**: Layer 1 is a platform-level prompt (constant across all agents — behavioral rules, safety guardrails, anti-hallucination instructions), Layer 2 is an industry-specific template (dental terminology, restaurant workflows, crisis protocols for psychology), and Layer 3 is business-specific content generated from the onboarding questionnaire. Prompts are versioned per agent per tenant, enabling A/B testing and instant rollback. The active version is marked `is_active = true`; updates create new version rows.

**Competitor insights shape key decisions.** Stammer.ai proves the Agency→Sub-Account two-tier multi-tenancy model, with agencies reselling at 3–5x markup on a prepaid wallet billing system. Stammer Aurora Inbox validates Mexican market requirements: MXN pricing, CFDI invoicing (critical for tax compliance), LFPDPPP data protection compliance, and a Spanish-first interface designed for non-technical users. Aurora Inbox Aurora's pricing tiers ($1,800–$6,000 MXN/month) establish viable price points. Aurora Inbox aurorainbox Stammer.ai's weakness — only OpenAI models at costs of $0.005–$0.01 per message Stammer — creates an opportunity to offer multi-model routing at lower costs through OpenRouter.

---

## 2. Questionnaire-driven onboarding that generates working agents in 10 minutes

The onboarding pipeline converts structured business answers into three artifacts simultaneously: a system prompt, a RAG knowledge base, and a voice agent configuration. When a dental clinic owner answers 10–15 questions about their services, hours, insurance, and emergency protocols, the system produces a fully functional WhatsApp chatbot and phone agent — no technical skill required.

**The pipeline architecture flows as follows**: industry selection loads the appropriate template → business identity data feeds the system prompt generator (role definition, personality, scope boundaries, behavioral rules) → structured business data feeds the RAG knowledge base builder (each service as one chunk, each FAQ as one chunk, hours/contact as atomic chunks) → voice/channel preferences feed the agent configuration generator (Retell AI parameters, WhatsApp API settings).

For each industry, specific questions prevent hallucination gaps:

**Dental offices** require: business name, dentist names, address, hours, services (checkboxes for cleaning through implants), price ranges or "consultation required" policy, insurance accepted (GNP, MetLife, Mapfre, Seguros Monterrey), appointment types, emergency protocol for after-hours, payment methods (cash, card, transfer, MSI monthly payments), and cancellation policy. **Restaurants** need: type (casual, taquería, fine dining), full menu with prices (upload or manual entry), reservation policy, delivery options (Uber Eats, Rappi, Didi Food), dietary accommodations, and parking info. **Psychologists** demand special attention: Cédula Profesional number, session types and therapeutic approaches, a mandatory confidentiality notice, and a **crisis protocol** — the chatbot must include the Línea de la Vida number (800-911-2000) when it detects suicidal ideation. **Salons** need stylist names and specialties, per-service pricing with duration, booking rules, and cancellation/no-show fees.

**Chunking strategy for RAG uses entity-based chunking** rather than document-based splitting, because business data is inherently structured and small. Each service becomes one chunk of **128–512 tokens** with metadata tags for category (services, faq, hours, policy, staff), industry type, and last-updated timestamp. FAQ entries are stored as single Q&A pair chunks. Prices, hours, and contact info are stored both as embeddings and in a separate `business_facts` table for deterministic SQL lookup — this dual storage is critical for anti-hallucination.

**Onboarding UX must be mobile-first** since 90%+ of Mexican SMBs will onboard via smartphone. The recommended flow is a **hybrid wizard + conversational approach**: a WhatsApp-native conversational welcome (2 minutes, covering industry selection and business name), then a structured 5–7 step wizard with progress bar (8–10 minutes, covering identity → hours → services → pricing → FAQs → policies), followed by instant gratification — a live preview of the agent responding to a sample question using the owner's actual data. Industry benchmarks show that **each extra form field reduces completion by 5–7%**, Saasfactor so signup should require only 3 fields: name, WhatsApp number, and business type. Target completion rate is 70%+ Alexander Jarvis by collecting only essential data first (agent goes live with basics), then using progressive disclosure via WhatsApp reminders: "Tu agente recibió 12 preguntas sobre precios esta semana. ¿Quieres agregar tus precios?"

A **Knowledge Coverage Score** gamifies completeness: 0–40% means frequent "I don't know" responses, 40–70% handles common questions, 70–90% is good, and 90–100% is fully equipped. This score drives both onboarding engagement and ongoing data enrichment.

---

## 3. Agent marketplace with event-driven triggers and composable workflows

The marketplace operates on a **Catalog + Registry architecture** where each pre-built agent is stored as an immutable JSON manifest containing system prompt templates, tool

configurations, trigger definitions, conversation flow graphs, required permissions, and a configuration schema defining what tenants can customize. When a business clicks "Activate," the system creates a `tenant_agent_instance` linking the tenant to the agent definition, copies defaults into a tenant-scoped configuration, provisions required resources (cron jobs, webhook listeners), and the agent goes live.

**Six core marketplace agents** form the initial catalog:

The **Collections Agent** triggers on `payment.overdue` events, executing a state machine: FIRST_REMINDER (friendly WhatsApp with payment link) → wait 48h → SECOND_REMINDER → wait 72h → ESCALATION (payment plan offer or human handoff) → THANK_YOU on payment receipt. It integrates with Conekta (Mexico's dominant payment gateway) and sends WhatsApp template messages via the Business API. Production platforms report **30% improvement in debt recovery** with this pattern. Aurora Inbox The **Review Requester** triggers 2 hours after `appointment.completed`, using sentiment detection to route positive responses to a direct Google Review URL and negative responses to internal feedback capture. The **Reactivation Agent** runs weekly via cron, querying customers inactive for 30+ days, segmenting by last service type and spend level, and sending personalized offers. The **NPS Survey Agent** achieves **2–3x higher completion rates** LivePerson than email surveys by running in-conversation on WhatsApp. The **Appointment Reminder** triggers at configurable hours before an appointment, using WhatsApp interactive buttons for confirm/reschedule/cancel, reducing no-shows by **20–30%.** LeadAgents The **Lead Qualification Agent** handles inbound WhatsApp messages from unknown numbers, qualifying through natural conversation (service interest, budget, timeline) and scoring leads for routing.

**The trigger system uses BullMQ** (Redis-backed) for scheduling, supporting cron-based triggers (daily review requests at 10am tenant-local-time), event-based triggers (`appointment.booked` → schedule reminder), and manual triggers (user clicks "Send Campaign"). BullMQ's Groups feature enables fair processing across tenants — each tenant is a group within shared queues. BullMQ For complex multi-day workflows like collections sequences with escalation logic, Temporal.io provides stateful orchestration, though BullMQ handles 90% of trigger needs.

**Configuration follows a three-layer merge**: platform defaults → agent definition defaults → tenant overrides, with later layers winning. Tenants can customize message templates, timing rules, business rules, escalation behavior, and offer amounts. The platform controls core conversation logic, security policies, WhatsApp compliance rules, and data retention. Each agent conversation is modeled as a finite state machine with states, transitions, and triggers stored as JSON in the agent definition, with per-conversation state tracked in the database.

**Versioning uses semver on agent definitions** that are never mutated — new versions create new rows. Minor version upgrades (2.0 → 2.1) auto-apply to tenant instances with new features using defaults. Major version upgrades (2.x → 3.0) require explicit tenant opt-in with a config migration tool. Running conversations always complete on their activated version.

## 4. Industry-specific dashboards powered by configuration, not code

The dashboard system uses **pre-configured industry templates with optional widget customization**, built on a configuration-driven architecture where industry differences live in JSON configs rather than code branches. Each tenant's `industry_type` field drives which dashboard template loads, and a `tenant_dashboard_config` table stores optional layout overrides.

**The recommended frontend stack is Tremor + Recharts on react-grid-layout.** Tremor (acquired by Vercel) provides 35+ production-ready dashboard components — BarChart, DonutChart, AreaChart, Tracker, BarList, and KPI cards — all built on React + Tailwind CSS + Radix UI with accessible keyboard navigation (DEV Community) and beautiful defaults. React-grid-layout (21K+ GitHub stars) supports responsive breakpoints, persisted widget positions, and per-widget drag/resize toggles for power users.

Metrics that matter vary dramatically by industry. **Dental dashboards** highlight appointments booked via AI, no-shows prevented (each costing **$80–$200 in Mexico**), (Arini) new patients from AI, after-hours appointments captured, and schedule fill rate. **Restaurant dashboards** focus on reservations handled by AI, order inquiries resolved without human intervention, review response rate, and table turnover. **Salon dashboards** track bookings via chatbot, cancellation rate change post-AI, product upsell rate from conversation recommendations, and stylist utilization (target 70–85%). (Me-Pos) **Psychology dashboards** emphasize intake forms completed, inquiry-to-first-session time, and cancellation rate (behavioral health averages exceed 5%).

**AI impact metrics calculate automatically from conversation data.** Messages saved = AI-handled conversations minus those escalated to humans (industry benchmark: **70–85% AI resolution rate**). (aurorainbox) Hours saved = messages saved × 10 minutes average human handling time ÷ 60. Money saved = hours saved × **100 MXN/hour** (the recommended default based on Mexican receptionist salaries of 10,000–15,000 MXN/month). (Aurora Inbox) These calculations require tracking five events in the conversation pipeline: `conversation_started`, `ai_responded`, `ai_resolved`, `escalated_to_human`, and `appointment_booked`.

**Pre-computed KPI tables** (hourly, daily, weekly, monthly aggregations) prevent expensive on-the-fly calculations. Background jobs aggregate conversation events into KPI tables, and dashboard API endpoints read from these tables per tenant. Widget lazy loading renders only visible components initially, and tenant-aware Redis caching includes `tenant_id` in cache keys.

---

## 5. ROI that justifies itself: formulas and Mexico-specific benchmarks

Aurora Inbox claims **2,000–5,000% ROI** for Mexican SMBs, and the math is internally

consistent if aggressive. Aurora Inbox Their calculation uses a very low denominator (platform cost of $99–$179/month) against comprehensive benefits (agent cost savings + revenue increase from faster response times + after-hours lead capture). A dental clinic handling 150 queries/month with 85% automation, cutting receptionist time from 4 hours/day to 45 minutes, and capturing 12 additional after-hours appointments per month at $200 each = **$2,880 benefit on $124 investment = 2,222% ROI**. Aurora Inbox aurorainbox

The **three-lever ROI model** works across industries: **Lever 1** is cost savings from agent load reduction (75–80% of queries automated × 10 min handling time × hourly cost), **Lever 2** is revenue increase from faster response Aurora Inbox (Harvard Business Review data shows **21× higher lead qualification** when responding within 5 minutes — AI responds in 3 seconds), and **Lever 3** is after-hours capture (35–45% of inquiries arrive outside business hours; without AI these go unanswered until next day, losing 70% of potential conversions). Aurora Inbox aurorainbox

**Real Mexico benchmarks for the ROI calculator**: small dental clinics receive **100–200 monthly inquiries** (5–8/day), restaurants **200–400** (8–15/day), salons **100–250** (4–10/day), real estate agencies **200–400** (8–15/day). Aurora Inbox uses 500 queries/month as their standard benchmark for businesses with established WhatsApp presence. Aurora Inbox aurorainbox Average receptionist salary in Mexico is **12,000–15,000 MXN/month** ($600–$750 USD), with an effective hourly rate including overhead of approximately **100 MXN/hour**. Dental no-shows cost **$80–$200** per missed appointment in Mexico (adjusted from US benchmarks of $200–$400). Each no-show per day translates to **$20,000–$50,000/year** in lost production. Arini

WhatsApp API costs are remarkably favorable for this model: **service conversations are free** Flowcall (customer-initiated messages within a 24-hour window), Flowcall utility templates cost **$0.0085/message**, YCloud and marketing templates cost **$0.0305/message** flowcall in Mexico. Since most SMB interactions are customer-initiated, the AI chatbot's responses cost **$0.00 in Meta fees** for the majority of conversations. An SMB handling 1,000 conversations/month pays roughly **$8.50–$30.50** in Meta API fees.

---

## 6. Unified customer timeline across WhatsApp and voice calls

The omnichannel architecture uses a **channel-agnostic `messages` table** where both WhatsApp text messages and voice call transcript segments are stored with a shared schema. The `content_type` field discriminates between `text`, `voice_segment`, `audio`, `image`, and `system_event`. The `channel` field marks `whatsapp` or `voice`. **Phone number serves as the primary identity key** across channels — WhatsApp accounts are tied to phone numbers, voice calls arrive with caller ID, and this overlap enables automatic identity linking without complex entity resolution.

When a voice call ends, Retell AI fires a `call_ended` webhook containing the full transcript (both as raw text and as timestamped `transcript_object` segments), recording URL, call

duration, and post-call analysis (summary, sentiment, custom extracted fields like customer name and intent). The ingestion flow looks up the customer by phone number, creates or updates the customer record, creates a `conversation` record with `channel = 'voice'`, and parses each transcript segment into individual `messages` rows. This means a dental patient who calls Tuesday morning to ask about pricing and then WhatsApp-messages Wednesday afternoon to book an appointment appears in **a single, chronological timeline** for the business owner.

**Respond.io's architecture validates this approach** — they serve as the most relevant competitor, offering a unified inbox where all conversations from WhatsApp, Facebook, Instagram, TikTok, web chat, email, and voice appear in a single view per business. Their key architectural pattern is automatic contact merging: when duplicate contacts from different channels are detected, they prompt merge into a unified profile with complete conversation history. Their event-driven backend handles massive cross-channel spikes with 99.999% uptime.

The customer timeline API uses **keyset pagination** (`WHERE created_at < ? ORDER BY created_at DESC LIMIT N`) instead of OFFSET for performance with large histories. Real-time updates leverage **Supabase Realtime** (WebSocket-based) to push new messages and voice call transcripts to the dashboard as they arrive, enabling live conversation monitoring across all channels.

---

## 7. Five-layer anti-hallucination defense for business-critical accuracy

Preventing a dental clinic's chatbot from inventing a price or fabricating a doctor's name requires defense in depth. RAG alone reduces hallucination by **60–80%**, aiqnahub AIQnAHUB but residual failures in retrieval (fetching topically relevant but factually wrong chunks) and generation (the LLM "fusing" information across documents) Mindee demand additional layers.

**Layer 1: Hybrid retrieval separates structured facts from semantic search.** Prices, hours, staff names, phone numbers, and addresses are stored in a `business_facts` table for deterministic SQL lookup — these must be **100% accurate** and never depend on embedding similarity. FAQ matching, service descriptions, and policy questions use pgvector cosine similarity search. An intent classifier routes each query: "price" intent → SQL lookup, "hours" intent → SQL lookup, "general" intent → vector search. Both paths run in parallel, with SQL results taking priority when available.

**Layer 2: System prompt engineering with repeated grounding instructions.** The prompt explicitly states at both the beginning and end: "ONLY answer using the information provided in the KNOWLEDGE BASE section. NEVER invent, guess, or extrapolate. When stating prices, hours, or contact info, include them EXACTLY as they appear — do not round, estimate, or modify." The fallback language is pre-written in Mexican Spanish: "No tengo esa información

disponible. ¿Te gustaría que te conecte con el equipo de [business_name]?" Temperature is set to **0.2–0.4** for factual queries, (ElevenLabs) reducing hallucination by approximately 22%. (aiqnahub)

**Layer 3: Confidence scoring with tiered fallback.** Retrieval confidence measures cosine similarity of the best matching chunk — below **0.70 threshold**, the knowledge base likely lacks the answer. Below **0.40**, the system always transfers to a human. Two consecutive low-confidence responses (<0.50 each) trigger automatic escalation. Frustration detection (Mexican Spanish markers: "esto no sirve", "quiero hablar con alguien", "ya me cansé") also triggers immediate human handoff.

**Layer 4: Output validation with Guardrails AI and regex.** Post-generation validation cross-references any MXN prices mentioned in the response against the `business_facts` table — if a price appears in the AI's response but doesn't exist in the database, the response is blocked and replaced with the fallback. Phone numbers are validated against stored formats. Schema enforcement via Pydantic catches structural errors. (toolhalla) NeMo Guardrails (NVIDIA) handles conversation flow control to keep the chatbot on-topic per business.

**Layer 5: Knowledge gap logging creates a feedback loop.** When the agent encounters unanswered questions, it logs them. The business owner sees a "Knowledge gaps to fill" dashboard showing "3 clientes preguntaron sobre estacionamiento esta semana — agrega esta info para mejorar tu agente." This progressively closes coverage gaps, and the Knowledge Coverage Score rises toward 100%.

**Embedding model selection matters for Spanish content.** Use `text-embedding-3-small` over the older ada-002 — it shows a **40% improvement on multilingual retrieval** (MIRACL benchmark: 44.0% vs 31.4%), is **5× cheaper** ($0.00002 vs $0.0001 per 1K tokens), and supports flexible dimensions (reduce to 512 for cost savings with minimal quality loss).

---

## 8. WhatsApp Business API with Embedded Signup for instant tenant onboarding

Meta's Embedded Signup (current version v4) enables businesses to connect their WhatsApp through your platform in a single popup flow. The user clicks "Connect WhatsApp," authenticates with Facebook credentials, accepts Terms of Service, selects or creates a Business Portfolio and WABA, verifies their phone number via OTP, and enters a display name. (facebook) (Medium) Upon completion, the SDK returns the **WABA ID**, **Phone Number ID**, and an exchangeable authorization code. (facebook)

**The backend flow after popup completion**: exchange the authorization code for a Business Integration System User access token (valid ~60 days) via `GET graph.facebook.com/v21.0/oauth/access_token`, (medium) register the phone number for Cloud API via `POST /{PHONE_NUMBER_ID}/register`, subscribe the app to WABA webhooks via `POST /{WABA_ID}/subscribed_apps`, (Medium) and store the credentials encrypted in the tenant record.

The 60-day token lifetime is a critical operational concern — implement expiry tracking with notifications 7 days before expiration, requiring re-authorization through a simplified re-signup flow. medium

**Multi-WABA management** uses the Tech Provider model: a single Meta Developer App, with each SMB tenant creating their own WABA through Embedded Signup. All webhooks for all WABAs route to your single callback URL by default. facebook 360dialog The webhook payload contains `phone_number_id` in the metadata, which serves as the primary routing key to the correct tenant. The routing algorithm is straightforward: extract `phone_number_id` from the payload, look up the tenant in the database, dispatch to the correct AI agent pipeline, and always return HTTP 200 immediately before processing asynchronously via a message queue.

**Rate limits for onboarding**: default is **10 new businesses per rolling 7 days**, increasing to **200/week** after completing Business Verification, App Review, and Access Verification. facebook New unverified tenants start at **250 messages/day** (Tier 0), Facebook Developers scaling to 1,000 after verification. Facebook Developers medium As of 2026, Meta is removing intermediate tiers — businesses scale directly from 1,000 to 100,000 daily unique users after verification. Sanuker Meta evaluates tier upgrades every **6 hours** Chatarmin when ⩾50% of current tier capacity is utilized with a "Green" quality rating. Medium

**Mexico pricing is exceptionally favorable**: service conversations (customer-initiated, within 24-hour window) are **completely free**, Aurora Inbox +2 utility messages cost $0.0085, YCloud and marketing messages cost $0.0305. flowcall Starting April 2026, Meta supports **MXN billing** natively, Authkey's Blog simplifying payment for Mexican SMBs. Authkey's Blog Template management requires submitting templates in `es_MX` locale, with up to 6,000 template translations per WABA and ML-assisted approval typically completed within minutes. Twilio

---

## 9. Voice agent deployment starting with Retell AI, scaling to custom infrastructure

**Phase 1 uses Retell AI's managed platform** for rapid deployment. Their REST API enables full programmatic lifecycle management: `POST /create-agent` with `response_engine` and `voice_id`, GitHub phone number provisioning and binding via `POST /create-phone-number` or `POST /import-phone-number` (for Telnyx/Twilio SIP imports), and per-call dynamic variable injection via `retell_llm_dynamic_variables` for tenant-specific greetings. Retell AI The critical configuration for Mexico: `language: "es-419"` (Latin American Spanish), ElevenLabs or Cartesia voices with Spanish support, Retell AI `backchannel_words: ["sí", "ajá", "claro", "entendido"]`, and `boosted_keywords` containing business-specific terms, Mexican street names, and neighborhood names.

Voice prompts differ fundamentally from chat prompts. Responses must be **60–70% shorter** VoiceInfra (target <20 words per turn), written as spoken language with no markdown or formatting, and include filler phrases ("Déjame verificar eso") before tool calls to mask

processing latency. Default to "usted" (formal) for business contexts — Mexican business culture expects formality. Turn-taking configuration uses `interruption_sensitivity: 0.8` LiveKit for medical/psychology (let patients interrupt) and `0.5` for sales contexts.

**Phone number provisioning through Telnyx** is recommended over Twilio for Mexico: Telnyx offers Mexican local numbers Telxi from **$1/number** Telnyx with SIP trunking at $0.005/min outbound Telnyx (vs Twilio's $0.010–$0.014/min), Telnyx HD voice codecs (G.722 at 16kHz critical for AI speech recognition), and a private MPLS network for lower latency. Telnyx Both require Mexican regulatory compliance documentation: Acta Constitutiva, authorized person's ID, latest invoice, and power of attorney if applicable. Telnyx

**Per-tenant voice architecture**: each tenant gets one or more agents created via API (separate agents for inbound support and outbound campaigns), dedicated phone number(s) bound to those agents, and per-tenant LLM configuration. Retell AI's weighted agent lists enable A/B testing: Retell AI Retell AI `inbound_agents: [{agent_id: "v1", weight: 0.8}, {agent_id: "v2", weight: 0.2}]`. Batch outbound campaigns use `POST /create-batch-call` with per-call dynamic variables (customer name, appointment date, business name). Voicemail detection (`voicemail_detection_timeout_ms: 30000`) automatically leaves configurable messages. Retell AI

**Phase 2 scales to Pipecat or LiveKit** for cost optimization above ~10,000 minutes/month. Pipecat (by Daily.co) is an open-source Python framework Pipecat Daily with 40+ AI service plugins, GitHub 300–600ms round-trip latency, Medium and deployment via Docker containers on Pipecat Cloud or self-hosted Kubernetes. LiveKit Agents LiveKit 1.0 offers multi-agent handoff, semantic turn detection LiveKit supporting Spanish (<25ms on CPU), and native SIP integration GitHub with Telnyx. LiveKit **All-in cost drops from $0.13–0.20/min (Retell) to $0.05–0.08/min** at scale with a custom stack (Deepgram STT at $0.15/hour + Cartesia TTS at $0.015/min + LLM via OpenRouter).

Retell AI's pricing: $0.07/min base (includes STT + LLM orchestration + base TTS), Retell AI premium voices add ~$0.015–$0.04/min, 20 free concurrent calls with additional at $8/concurrent call/month. Retell AI The **breakpoint for migration** to self-hosted is approximately 10,000 minutes/month, where infrastructure investment pays back in per-minute savings.

---

## 10. OpenRouter as the intelligent model routing backbone

OpenRouter provides access to **290+ AI models** through a single OpenAI-compatible API endpoint, with automatic provider failover, zero-completion insurance (billed only for successful runs), and ~25–40ms latency overhead. The pricing model passes through provider costs at zero markup on inference, with a **5.5% platform fee on credit purchases** Skywork — negligible compared to the engineering cost of maintaining multiple provider integrations.

**The recommended three-layer routing architecture** dramatically reduces costs while maintaining quality. Layer 1 is a lightweight task classifier (GPT-5 nano at $0.05/M input tokens or Gemini Flash-Lite at $0.25/M) that detects intent and complexity. Layer 2 routes to the appropriate model tier: fast tier (simple queries → DeepSeek V3.2 at $0.14/M or Gemini Flash at $0.50/M), balanced tier (general conversation → Claude Haiku 4.5 at $1/M or GPT-4.1 at $2/M), or high-quality tier (sensitive topics, complaints → Claude Sonnet 4.5 at $3/M). LMSYS RouteLLM research shows this cascading approach **reduces costs by 85% while maintaining 95% of top-model performance**. One customer support platform documented a reduction from $42,000 to $18,000/month.

**For voice agents requiring sub-500ms latency**, use OpenRouter's `:nitro` suffix to auto-route to the fastest available provider. OpenRouter Groq-hosted Llama 4 Maverick delivers consistent **200ms processing** with excellent Spanish performance, making it ideal for real-time voice. The complete voice latency stack: Deepgram Nova-3 STT (~100ms) + LLM via OpenRouter `:nitro` (~200ms TTFT) + Cartesia/Kokoro TTS (~100ms) = **~400–500ms total**, well within the conversational threshold.

**Prompt caching delivers the highest-ROI cost optimization.** OpenAI automatically caches prompts >1,024 tokens with cache hits at **10% of standard cost** (90% discount). Anthropic requires explicit `cache_control` headers with a 5-minute TTL. For application-level caching, implement three tiers: L1 in-memory exact match, L2 Redis shared normalized match, and L3 semantic cache via vector DB (>0.90 similarity threshold). Medium Combined with routing, target **$0.003–$0.005 per conversation** at scale — roughly **$28–50/month for 10,000 conversations** across all tenants.

**Per-tenant model configuration maps to subscription tiers.** A free tier uses DeepSeek V3.2 / free models with a 500K token monthly budget and 100 conversations. The basic tier ($29/month) upgrades to Claude Haiku 4.5 with 5M tokens and 1,000 conversations. The premium tier ($99/month) uses Claude Sonnet 4.5 for sensitive topics, full fallback chains, and semantic caching with 25M tokens and 10,000 conversations. OpenRouter Presets — named configurations encapsulating model selection, provider routing, and system prompts — enable configuration changes without code deployments: OpenRouter create presets like "smb-free-tier" and "smb-premium" and reference them by name in API calls.

For observability, **Langfuse** (open-source, self-hostable) provides model-agnostic tracing, prompt management, and quality evaluations, while **Helicone** offers proxy-based integration with negligible latency overhead and built-in caching. Track token usage per tenant via OpenRouter's Activity API, implement middleware that checks remaining budget before forwarding requests, and auto-downgrade model quality when approaching limits (Sonnet → Haiku → DeepSeek).

---

## Conclusion: from blueprint to production

This platform sits at the intersection of three converging trends: WhatsApp's dominance in Mexican business Aurora Inbox communication (**71% of digital businesses sell via WhatsApp**), the rapid cost decline of AI inference (making per-conversation costs negligible at $0.003–$0.005), and the Mexican SMB market's acute need for automation (receptionist salaries of 12,000–15,000 MXN/month for work that AI handles 75–85% of the time). The result is an ROI proposition that genuinely delivers 2,000%+ returns for engaged businesses.

**The critical architectural decisions that determine success**: Supabase with RLS for multi-tenancy (not separate databases), hybrid SQL + vector retrieval for zero-hallucination on business facts, BullMQ for marketplace trigger orchestration, Retell AI for Phase 1 voice with Telnyx for Mexican telephony, OpenRouter with cascading model routing for cost optimization, and a questionnaire-driven onboarding flow that targets 70%+ completion through mobile-first Spanish-first design with progressive disclosure.

The **build sequence should prioritize** the WhatsApp chat agent with anti-hallucination (highest market demand, lowest infrastructure complexity), then the onboarding questionnaire pipeline (converts signups to working agents), then the voice agent layer (differentiator from Aurora Inbox and most competitors), then the marketplace agents (collections, reminders, reviews — these drive retention), and finally the ROI dashboard (proves value, reduces churn). Each layer builds on the shared multi-tenant foundation, and the architecture supports starting with 10 tenants on a single Supabase instance and scaling to thousands without re-architecture.